Projet données 2025

Zeying QU, Chung-Anh NGUYEN, Julian GRALL

2025-11-02

Credit Scoring project

Présentation des données et nettoyage

Présentation du jeu de données

Le jeu de données utilisé dans cette étude provient de la base publique *Default of Credit Card Clients Dataset* (UCI / Kaggle). Il contient des informations sur 30 000 clients de cartes de crédit à Taïwan, collectées en 2025, dans le but de prédire le risque de défaut de paiement pour le mois suivant.

Chaque observation correspond à un client, et les variables décrivent ses caractéristiques démographiques, son historique de paiement, ses montants de factures et ses paiements récents.

La variable cible (`default.payment.next.month`) prend la valeur **1** si le client n'a pas remboursé son crédit à la date prévue, et **0** sinon.

```r
data <- readr::read_csv("UCI_Credit_Card_ZeyingQU_AnhChungNGUYEN_JulianGRALL.csv")
```

```
## Rows: 30000 Columns: 25
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (25): ID, LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE, PAY_0, PAY_2, PAY_3,...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
#Aperçu des premières lignes
head(data)
```

```
## # A tibble: 6 x 25
##      ID LIMIT_BAL   SEX EDUCATION MARRIAGE   AGE PAY_0 PAY_2 PAY_3 PAY_4 PAY_5
##   <dbl>     <dbl> <dbl>     <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     1     20000     2         2        1    24     2     2    -1    -1    -2
## 2     2    120000     2         2        2    26    -1     2     0     0     0
## 3     3     90000     2         2        2    34     0     0     0     0     0
## 4     4     50000     2         2        1    37     0     0     0     0     0
## 5     5     50000     1         2        1    57    -1     0    -1     0     0
## 6     6     50000     1         1        2    37     0     0     0     0     0
## # i 14 more variables: PAY_6 <dbl>, BILL_AMT1 <dbl>, BILL_AMT2 <dbl>,
## #   BILL_AMT3 <dbl>, BILL_AMT4 <dbl>, BILL_AMT5 <dbl>, BILL_AMT6 <dbl>,
## #   PAY_AMT1 <dbl>, PAY_AMT2 <dbl>, PAY_AMT3 <dbl>, PAY_AMT4 <dbl>,
## #   PAY_AMT5 <dbl>, PAY_AMT6 <dbl>, default.payment.next.month <dbl>
```

```r
#Dimensions du dataset
dim(data)
```

```
## [1] 30000    25
```

Le jeu de données comporte 30 000 observations de 25 variables.

Variable	Description
ID	Identifiant de chaque client
LIMIT_BAL	Montant du crédit accordé en dollars taïwanais (NT$), incluant le crédit individuel et familial/supplémentaire
SEX	Sexe (1 = homme, 2 = femme)
EDUCATION	Niveau d'éducation (1 = école supérieure, 2 = université, 3 = lycée, 4 = autres, 5 = inconnu, 6 = inconnu)
MARRIAGE	Statut marital (1 = marié, 2 = célibataire, 3 = autres)
AGE	Âge en années
PAY_0	Statut de remboursement en septembre 2005 (-1 = paiement effectué à temps, 1 = retard d'un mois, 2 = retard de deux mois, …, 8 = retard de huit mois, 9 = retard de neuf mois ou plus)
PAY_2	Statut de remboursement en août 2005 (échelle identique à ci-dessus)
PAY_3	Statut de remboursement en juillet 2005 (échelle identique à ci-dessus)
PAY_4	Statut de remboursement en juin 2005 (échelle identique à ci-dessus)
PAY_5	Statut de remboursement en mai 2005 (échelle identique à ci-dessus)
PAY_6	Statut de remboursement en avril 2005 (échelle identique à ci-dessus)
BILL_AMT1	Montant de la facture en septembre 2005 (NT$)
BILL_AMT2	Montant de la facture en août 2005 (NT$)
BILL_AMT3	Montant de la facture en juillet 2005 (NT$)
BILL_AMT4	Montant de la facture en juin 2005 (NT$)
BILL_AMT5	Montant de la facture en mai 2005 (NT$)
BILL_AMT6	Montant de la facture en avril 2005 (NT$)
PAY_AMT1	Montant du paiement précédent en septembre 2005 (NT$)
PAY_AMT2	Montant du paiement précédent en août 2005 (NT$)
PAY_AMT3	Montant du paiement précédent en juillet 2005 (NT$)
PAY_AMT4	Montant du paiement précédent en juin 2005 (NT$)
PAY_AMT5	Montant du paiement précédent en mai 2005 (NT$)
PAY_AMT6	Montant du paiement précédent en avril 2005 (NT$)
default.payment.next.month	Défaut de paiement le mois suivant (1 = oui, 0 = non)

```r
table(data$default.payment.next.month)
```

```
##
##     0     1
## 23364  6636
```

```r
prop.table(table(data$default.payment.next.month))
```

```
##
##      0      1
## 0.7788 0.2212
```

On observe que 22% des clients sont en défaut de paiement, ce qui indique un déséquilibre de classes à prendre en compte lors de la modélisation.

Nettoyage du jeu de données

```r
colSums(is.na(data))
```

Vérifier les valeurs manquantes

```
##                          ID                   LIMIT_BAL
##                           0                           0
##                         SEX                   EDUCATION
##                           0                           0
##                    MARRIAGE                         AGE
##                           0                           0
##                       PAY_0                       PAY_2
##                           0                           0
##                       PAY_3                       PAY_4
##                           0                           0
##                       PAY_5                       PAY_6
##                           0                           0
##                   BILL_AMT1                   BILL_AMT2
##                           0                           0
##                   BILL_AMT3                   BILL_AMT4
##                           0                           0
##                   BILL_AMT5                   BILL_AMT6
##                           0                           0
##                    PAY_AMT1                    PAY_AMT2
##                           0                           0
##                    PAY_AMT3                    PAY_AMT4
##                           0                           0
##                    PAY_AMT5                    PAY_AMT6
##                           0                           0
## default.payment.next.month
##                           0
```

On remarque qu'il n'y a aucune valeur manquante

```r
data <- data %>% select(-ID)
```

Supprimer la colonne inutile La colonne `ID` ne contient qu'un identifiant, donc elle n'apporte aucune information à la modélisation.

Les valeurs incohérentes Maintenant corrigeons les valeurs incohérentes.

```r
table(data$EDUCATION)
```

```
##
##     0     1     2     3     4     5     6
##    14 10585 14030  4917   123   280    51
```

On remarque que dans la variable `EDUCATION` les valeurs **5** et **6** veulent dire la même chose. De plus elle peut prendre la valeur **0** ce qui semble être une erreur. Regroupons les dans **5**.

```r
data$EDUCATION[data$EDUCATION == 0 | data$EDUCATION == 6] <- 5
```

```r
unique(data$MARRIAGE)
```

```
## [1] 1 2 3 0
```

La variable `MARRIAGE` peut prendre la valeur **0** ce qui semble être une erreur, mettons les dans **3** (autres).

```r
data$MARRIAGE[data$MARRIAGE == 0] <- 3
```

Les types de variables Vérifions les types de variables

```r
str(data)
```

```
## tibble [30,000 x 24] (S3: tbl_df/tbl/data.frame)
##  $ LIMIT_BAL                 : num [1:30000] 20000 120000 90000 50000 50000 50000 500000 100000 1400
##  $ SEX                       : num [1:30000] 2 2 2 2 1 1 1 2 2 1 ...
##  $ EDUCATION                 : num [1:30000] 2 2 2 2 2 1 1 2 3 3 ...
##  $ MARRIAGE                  : num [1:30000] 1 2 2 1 1 2 2 2 1 2 ...
##  $ AGE                       : num [1:30000] 24 26 34 37 57 37 29 23 28 35 ...
##  $ PAY_0                     : num [1:30000] 2 -1 0 0 -1 0 0 0 0 -2 ...
##  $ PAY_2                     : num [1:30000] 2 2 0 0 0 0 0 -1 0 -2 ...
##  $ PAY_3                     : num [1:30000] -1 0 0 0 -1 0 0 -1 2 -2 ...
##  $ PAY_4                     : num [1:30000] -1 0 0 0 0 0 0 0 0 -2 ...
##  $ PAY_5                     : num [1:30000] -2 0 0 0 0 0 0 0 0 -1 ...
##  $ PAY_6                     : num [1:30000] -2 2 0 0 0 0 0 -1 0 -1 ...
##  $ BILL_AMT1                 : num [1:30000] 3913 2682 29239 46990 8617 ...
##  $ BILL_AMT2                 : num [1:30000] 3102 1725 14027 48233 5670 ...
##  $ BILL_AMT3                 : num [1:30000] 689 2682 13559 49291 35835 ...
##  $ BILL_AMT4                 : num [1:30000] 0 3272 14331 28314 20940 ...
##  $ BILL_AMT5                 : num [1:30000] 0 3455 14948 28959 19146 ...
##  $ BILL_AMT6                 : num [1:30000] 0 3261 15549 29547 19131 ...
##  $ PAY_AMT1                  : num [1:30000] 0 0 1518 2000 2000 ...
##  $ PAY_AMT2                  : num [1:30000] 689 1000 1500 2019 36681 ...
##  $ PAY_AMT3                  : num [1:30000] 0 1000 1000 1200 10000 657 38000 0 432 0 ...
##  $ PAY_AMT4                  : num [1:30000] 0 1000 1000 1100 9000 ...
##  $ PAY_AMT5                  : num [1:30000] 0 0 1000 1069 689 ...
##  $ PAY_AMT6                  : num [1:30000] 0 2000 5000 1000 679 ...
##  $ default.payment.next.month: num [1:30000] 1 1 0 0 0 0 0 0 0 0 ...
```

On remarque que toutes les variables sont en *numeric* alors que certaines d'entre elles devraient être des *factors*, comme `MARRIAGE`, `EDUCATION`, `SEX`…

Convertissons les variables qualitatives en *factors*

```
data$SEX <- as.factor(data$SEX)
data$EDUCATION <- as.factor(data$EDUCATION)
data$MARRIAGE <- as.factor(data$MARRIAGE)
data$default.payment.next.month <- as.factor(data$default.payment.next.month)
```

Ce sera utile pour la régression logistique et d'autres modèles, pour que R comprenne que ces variables ne sont pas continues.

Voilà notre jeu de données final :

```
head(data)
```

```
## # A tibble: 6 x 24
##   LIMIT_BAL SEX   EDUCATION MARRIAGE   AGE PAY_0 PAY_2 PAY_3 PAY_4 PAY_5 PAY_6
##       <dbl> <fct> <fct>     <fct>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     20000 2     2         1           24     2     2    -1    -1    -2    -2
## 2    120000 2     2         2           26    -1     2     0     0     0     2
## 3     90000 2     2         2           34     0     0     0     0     0     0
## 4     50000 2     2         1           37     0     0     0     0     0     0
## 5     50000 1     2         1           57    -1     0    -1     0     0     0
## 6     50000 1     1         2           37     0     0     0     0     0     0
## # i 13 more variables: BILL_AMT1 <dbl>, BILL_AMT2 <dbl>, BILL_AMT3 <dbl>,
## #   BILL_AMT4 <dbl>, BILL_AMT5 <dbl>, BILL_AMT6 <dbl>, PAY_AMT1 <dbl>,
## #   PAY_AMT2 <dbl>, PAY_AMT3 <dbl>, PAY_AMT4 <dbl>, PAY_AMT5 <dbl>,
## #   PAY_AMT6 <dbl>, default.payment.next.month <fct>
```

```
summary(data)
```

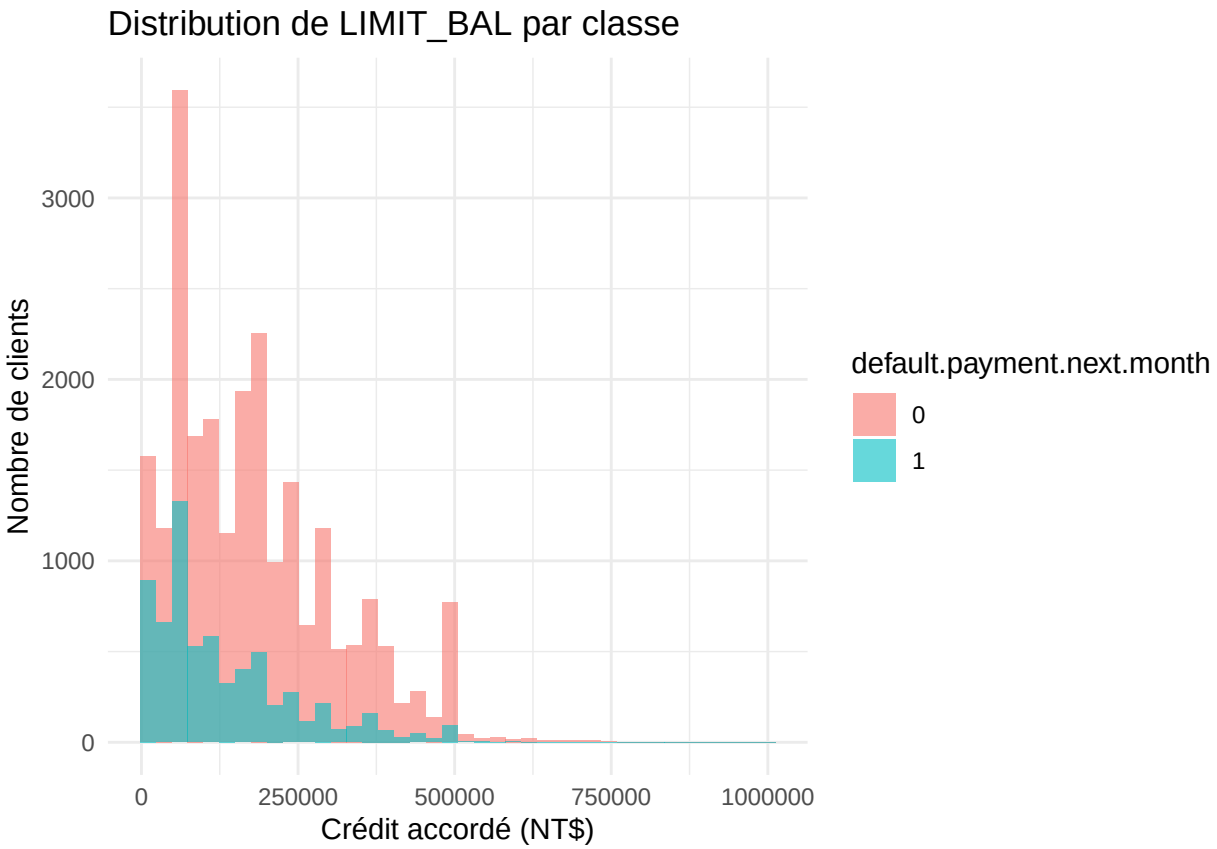
```
##    LIMIT_BAL       SEX       EDUCATION MARRIAGE       AGE
##  Min.   :  10000   1:11888   1:10585   1:13659   Min.   :21.00
##  1st Qu.:  50000   2:18112   2:14030   2:15964   1st Qu.:28.00
##  Median : 140000             3: 4917   3:  377   Median :34.00
##  Mean   : 167484             4:  123             Mean   :35.49
##  3rd Qu.: 240000             5:  345             3rd Qu.:41.00
##  Max.   :1000000                                 Max.   :79.00
##      PAY_0             PAY_2             PAY_3             PAY_4
##  Min.   :-2.0000   Min.   :-2.0000   Min.   :-2.0000   Min.   :-2.0000
##  1st Qu.:-1.0000   1st Qu.:-1.0000   1st Qu.:-1.0000   1st Qu.:-1.0000
##  Median : 0.0000   Median : 0.0000   Median : 0.0000   Median : 0.0000
##  Mean   :-0.0167   Mean   :-0.1338   Mean   :-0.1662   Mean   :-0.2207
##  3rd Qu.: 0.0000   3rd Qu.: 0.0000   3rd Qu.: 0.0000   3rd Qu.: 0.0000
##  Max.   : 8.0000   Max.   : 8.0000   Max.   : 8.0000   Max.   : 8.0000
##      PAY_5             PAY_6           BILL_AMT1         BILL_AMT2
##  Min.   :-2.0000   Min.   :-2.0000   Min.   :-165580   Min.   :-69777
##  1st Qu.:-1.0000   1st Qu.:-1.0000   1st Qu.:   3559   1st Qu.:  2985
##  Median : 0.0000   Median : 0.0000   Median :  22382   Median : 21200
##  Mean   :-0.2662   Mean   :-0.2911   Mean   :  51223   Mean   : 49179
##  3rd Qu.: 0.0000   3rd Qu.: 0.0000   3rd Qu.:  67091   3rd Qu.: 64006
```

```
##  Max.   : 8.0000   Max.   : 8.0000   Max.   : 964511   Max.   :983931
##    BILL_AMT3         BILL_AMT4         BILL_AMT5         BILL_AMT6
##  Min.   :-157264   Min.   :-170000   Min.   :-81334   Min.   :-339603
##  1st Qu.:   2666   1st Qu.:   2327   1st Qu.:  1763   1st Qu.:   1256
##  Median :  20089   Median :  19052   Median : 18105   Median :  17071
##  Mean   :  47013   Mean   :  43263   Mean   : 40311   Mean   :  38872
##  3rd Qu.:  60165   3rd Qu.:  54506   3rd Qu.: 50191   3rd Qu.:  49198
##  Max.   :1664089   Max.   : 891586   Max.   :927171   Max.   : 961664
##     PAY_AMT1         PAY_AMT2          PAY_AMT3         PAY_AMT4
##  Min.   :     0   Min.   :      0   Min.   :     0   Min.   :     0
##  1st Qu.:  1000   1st Qu.:    833   1st Qu.:   390   1st Qu.:   296
##  Median :  2100   Median :   2009   Median :  1800   Median :  1500
##  Mean   :  5664   Mean   :   5921   Mean   :  5226   Mean   :  4826
##  3rd Qu.:  5006   3rd Qu.:   5000   3rd Qu.:  4505   3rd Qu.:  4013
##  Max.   :873552   Max.   :1684259   Max.   :896040   Max.   :621000
##     PAY_AMT5           PAY_AMT6        default.payment.next.month
##  Min.   :     0.0   Min.   :     0.0   0:23364
##  1st Qu.:   252.5   1st Qu.:   117.8   1: 6636
##  Median :  1500.0   Median :  1500.0
##  Mean   :  4799.4   Mean   :  5215.5
##  3rd Qu.:  4031.5   3rd Qu.:  4000.0
##  Max.   :426529.0   Max.   :528666.0
```

Première vu du jeu données

1. Répartition des limites de crédit

```r
ggplot(data, aes(LIMIT_BAL, fill = default.payment.next.month)) +
  geom_histogram(bins = 40, alpha = 0.6, position = "identity") +
  labs(
    title = "Distribution de LIMIT_BAL par classe",
    x = "Crédit accordé (NT$)",
    y = "Nombre de clients"
  ) +
  theme_minimal()
```

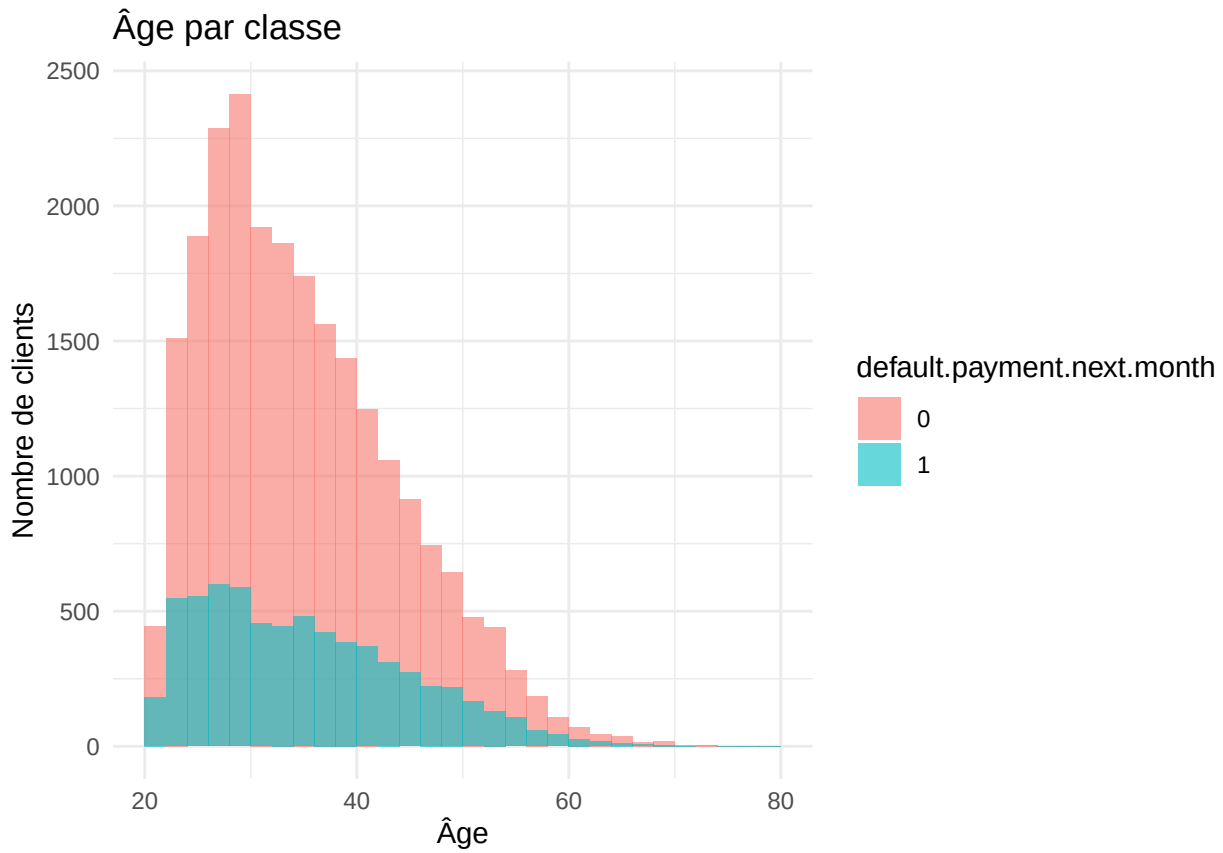


Nous pouvons observer que la majorité des limites de crédit des clients se concentrent dans une fourchette comprise entre 0 et 250 000 NT ; dans cette fourchette, le nombre de clients non défaillants dépasse largement celui des clients défaillants. Lorsque les limites de crédit dépassent 500 000 NT, il n'y a pratiquement aucun client défaillant.

Les clients disposant de limites de crédit plus élevées sont plus susceptibles de rembourser dans les délais, tandis que ceux dont les limites sont plus faibles présentent un risque de défaut de paiement plus important.

2. Répartition par âge

```
ggplot(data, aes(AGE, fill = default.payment.next.month)) +
  geom_histogram(bins = 30, alpha = 0.6, position = "identity") +
  labs(title = "Âge par classe", x = "Âge", y = "Nombre de clients") +
  theme_minimal()
```



Nous pouvons observer que les clients détenteurs d'une carte de crédit sont principalement concentrés dans la tranche d'âge des 20 à 40 ans ; les clients plus jeunes (20 ans) présentent un taux de défaut de paiement plus élevé ; avec l'âge, le nombre de défaillants diminue sensiblement ; au-delà de 50 ans, les clients défaillants et non défaillants deviennent relativement rares.

Les jeunes sont plus enclins à ne pas rembourser leurs dettes de carte de crédit, tandis que les personnes plus âgées font preuve d'une plus grande stabilité dans leurs remboursements.

3. Analyse des corrélations entre les variables numériques

```r
num_vars <- select_if(data, is.numeric)
cor(num_vars, use = "pairwise.complete.obs")[1:6, 1:6]
```

```
##            LIMIT_BAL         AGE       PAY_0       PAY_2       PAY_3
## LIMIT_BAL  1.0000000  0.14471280 -0.27121433 -0.29638210 -0.28612295
## AGE        0.1447128  1.00000000 -0.03944738 -0.05014778 -0.05304844
## PAY_0     -0.2712143 -0.03944738  1.00000000  0.67216438  0.57424509
## PAY_2     -0.2963821 -0.05014778  0.67216438  1.00000000  0.76655168
## PAY_3     -0.2861230 -0.05304844  0.57424509  0.76655168  1.00000000
## PAY_4     -0.2674600 -0.04972167  0.53884063  0.66206713  0.77735887
##                 PAY_4
## LIMIT_BAL -0.26746001
## AGE       -0.04972167
## PAY_0      0.53884063
## PAY_2      0.66206713
```

```
## PAY_3      0.77735887
## PAY_4      1.00000000
```

PAY_0, PAY_2 et PAY_3 présentent un degré élevé de corrélation, ce qui indique que les habitudes de remboursement des clients restent constantes d'un mois à l'autre.

LIMIT_BAL présente une corrélation négative avec le paiement, ce qui signifie que des limites de crédit plus élevées sont corrélées à une probabilité plus faible de retards de paiement.

L'âge présente une faible corrélation avec les autres variables et n'aide que de manière limitée à prédire les défauts de paiement.

Première analyse du jeu de données - Régression logistique

Introduction théorique : La régression logistique est un modèle statistique permettant d'estimer la probabilité qu'un événement binaire se produise (ici : défaut de paiement = 1).

Contrairement à la régression linéaire, elle modélise la relation entre les variables explicatives et la log-odds de l'événement. La fonction logit garantit que les probabilités prédites sont comprises entre 0 et 1.

1. Séparation du jeu de données

```r
set.seed(123)
n <- nrow(data)
id_train <- sample(1:n, size = 0.7 * n)
train <- data[id_train, ]
test <- data[-id_train, ]
cat("Proportion de défaut dans train:", mean(train$default.payment.next.month == "1"), "\n")
```

```
## Proportion de défaut dans train: 0.2228095
```

```r
cat("Proportion de défaut dans test:", mean(test$default.payment.next.month == "1"), "\n")
```

```
## Proportion de défaut dans test: 0.2174444
```

2. Modèle logistique complet

```r
modele_complet <- glm(default.payment.next.month ~ ., data = train, family = binomial)
summary(modele_complet)
```

```
##
## Call:
## glm(formula = default.payment.next.month ~ ., family = binomial,
##     data = train)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -9.968e-01  1.036e-01  -9.618  < 2e-16 ***
## LIMIT_BAL   -8.365e-07  1.910e-07  -4.380 1.19e-05 ***
```

```
## SEX2        -1.321e-01  3.677e-02  -3.592 0.000329 ***
## EDUCATION2  -8.353e-02  4.253e-02  -1.964 0.049530 *
## EDUCATION3  -1.642e-01  5.725e-02  -2.868 0.004128 **
## EDUCATION4  -1.092e+00  4.700e-01  -2.324 0.020146 *
## EDUCATION5  -1.354e+00  2.674e-01  -5.063 4.14e-07 ***
## MARRIAGE2   -1.758e-01  4.143e-02  -4.244 2.19e-05 ***
## MARRIAGE3   -1.104e-01  1.568e-01  -0.704 0.481474
## AGE          6.761e-03  2.234e-03   3.026 0.002480 **
## PAY_0        5.880e-01  2.123e-02  27.697  < 2e-16 ***
## PAY_2        8.710e-02  2.405e-02   3.622 0.000293 ***
## PAY_3        5.733e-02  2.712e-02   2.114 0.034534 *
## PAY_4        4.559e-02  3.007e-02   1.516 0.129469
## PAY_5        7.390e-03  3.241e-02   0.228 0.819645
## PAY_6        1.592e-02  2.668e-02   0.597 0.550680
## BILL_AMT1   -6.188e-06  1.354e-06  -4.570 4.87e-06 ***
## BILL_AMT2    2.531e-06  1.782e-06   1.420 0.155569
## BILL_AMT3    2.471e-06  1.586e-06   1.558 0.119242
## BILL_AMT4    2.449e-07  1.611e-06   0.152 0.879199
## BILL_AMT5    7.505e-07  1.884e-06   0.398 0.690273
## BILL_AMT6   -3.805e-07  1.479e-06  -0.257 0.797021
## PAY_AMT1    -1.487e-05  2.875e-06  -5.173 2.31e-07 ***
## PAY_AMT2    -1.145e-05  2.626e-06  -4.361 1.29e-05 ***
## PAY_AMT3    -8.983e-07  1.973e-06  -0.455 0.648846
## PAY_AMT4    -5.495e-06  2.314e-06  -2.374 0.017585 *
## PAY_AMT5    -4.178e-06  2.231e-06  -1.873 0.061092 .
## PAY_AMT6    -3.895e-06  1.668e-06  -2.335 0.019545 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 22279  on 20999  degrees of freedom
## Residual deviance: 19435  on 20972  degrees of freedom
## AIC: 19491
##
## Number of Fisher Scoring iterations: 6
```

Nous utilisons *glm()* avec *family = binomial* car la variable réponse `default.payment.next.month` est binaire (0 = pas de défaut, 1 = défaut).

3. Sélection de variables - Critère AIC

Sélection backward (on supprime les variables les moins pertinentes)

```r
modele_reduit <- step(modele_complet, direction = "backward", trace = FALSE)
summary(modele_reduit)
```

```
##
## Call:
## glm(formula = default.payment.next.month ~ LIMIT_BAL + SEX +
##     EDUCATION + MARRIAGE + AGE + PAY_0 + PAY_2 + PAY_3 + PAY_4 +
##     BILL_AMT1 + BILL_AMT2 + BILL_AMT3 + PAY_AMT1 + PAY_AMT2 +
##     PAY_AMT4 + PAY_AMT5 + PAY_AMT6, family = binomial, data = train)
```

```
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -9.992e-01  1.036e-01  -9.645  < 2e-16 ***
## LIMIT_BAL   -8.431e-07  1.890e-07  -4.461 8.15e-06 ***
## SEX2        -1.317e-01  3.676e-02  -3.583  0.00034 ***
## EDUCATION2  -8.412e-02  4.252e-02  -1.979  0.04787 *
## EDUCATION3  -1.658e-01  5.722e-02  -2.898  0.00375 **
## EDUCATION4  -1.093e+00  4.700e-01  -2.326  0.02000 *
## EDUCATION5  -1.358e+00  2.672e-01  -5.083 3.72e-07 ***
## MARRIAGE2   -1.760e-01  4.143e-02  -4.248 2.16e-05 ***
## MARRIAGE3   -1.128e-01  1.567e-01  -0.720  0.47170
## AGE          6.739e-03  2.234e-03   3.017  0.00256 **
## PAY_0        5.895e-01  2.118e-02  27.835  < 2e-16 ***
## PAY_2        8.900e-02  2.393e-02   3.719  0.00020 ***
## PAY_3        5.879e-02  2.702e-02   2.176  0.02958 *
## PAY_4        6.014e-02  2.384e-02   2.523  0.01164 *
## BILL_AMT1   -6.254e-06  1.353e-06  -4.622 3.79e-06 ***
## BILL_AMT2    2.558e-06  1.778e-06   1.439  0.15027
## BILL_AMT3    3.065e-06  1.238e-06   2.476  0.01327 *
## PAY_AMT1    -1.502e-05  2.862e-06  -5.249 1.53e-07 ***
## PAY_AMT2    -1.170e-05  2.598e-06  -4.502 6.74e-06 ***
## PAY_AMT4    -5.077e-06  2.026e-06  -2.506  0.01220 *
## PAY_AMT5    -4.624e-06  1.909e-06  -2.422  0.01543 *
## PAY_AMT6    -3.919e-06  1.649e-06  -2.377  0.01747 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 22279  on 20999  degrees of freedom
## Residual deviance: 19436  on 20978  degrees of freedom
## AIC: 19480
##
## Number of Fisher Scoring iterations: 6
```

Comparaison des AIC

```r
AIC(modele_complet, modele_reduit)
```

```
##                df      AIC
## modele_complet 28 19490.53
## modele_reduit  22 19479.96
```

Le modèle réduit obtenu par sélection automatique *step()* a un AIC inférieur au modèle complet, ce qui indique qu'un modèle plus simple ajuste les données tout aussi bien. Les variables supprimées apportaient peu d'information pour expliquer le défaut de paiement, et leur retrait améliore la parcimonie et l'interprétabilité du modèle.

4. Interprétation des coefficients

Odds ratios (exp des coefficients)

Dans une régression logistique, les coefficients sont sur l'échelle du logit et difficiles à interpréter directement. En exponentiant les coefficients, on obtient des multiplicateurs de chances, qui indiquent combien la probabilité de l'événement augmente ou diminue pour une unité de variation de la variable.

```
odd_ratios <- exp(coef(modele_reduit))
odd_ratios
```

```
## (Intercept)   LIMIT_BAL        SEX2  EDUCATION2  EDUCATION3  EDUCATION4
##   0.3681806   0.9999992   0.8766149   0.9193216   0.8471993   0.3351128
##  EDUCATION5   MARRIAGE2   MARRIAGE3         AGE       PAY_0       PAY_2
##   0.2571075   0.8386483   0.8933316   1.0067619   1.8030690   1.0930820
##       PAY_3       PAY_4   BILL_AMT1   BILL_AMT2   BILL_AMT3    PAY_AMT1
##   1.0605504   1.0619894   0.9999937   1.0000026   1.0000031   0.9999850
##    PAY_AMT2    PAY_AMT4    PAY_AMT5    PAY_AMT6
##   0.9999883   0.9999949   0.9999954   0.9999961
```

```
odd_ratios[odd_ratios > 1.1 | odd_ratios < 0.9]
```

```
## (Intercept)        SEX2  EDUCATION3  EDUCATION4  EDUCATION5   MARRIAGE2
##   0.3681806   0.8766149   0.8471993   0.3351128   0.2571075   0.8386483
##   MARRIAGE3       PAY_0
##   0.8933316   1.8030690
```

5. Évaluation sur les données test

```
proba_test <- predict(modele_reduit, newdata = test, type = "response")
head(proba_test)
```

```
##          1          2          3          4          5          6
## 0.53630052 0.24605822 0.17235401 0.07294851 0.04191465 0.21795640
```

Dans un premier temps, nous utilisons un seuil de classification de 0,5 car c'est le seuil standard pour transformer les probabilités prédites par une régression logistique en prédictions binaires. Concrètement, si la probabilité prédite d'un défaut est supérieure à 0,5, le client est classé comme défaillant, sinon comme non-défaillant. Ce seuil permet une première évaluation simple et équilibrée du modèle avant d'envisager un ajustement pour optimiser la sensibilité ou la précision selon les besoins.

```
pred_test <- ifelse(proba_test > 0.5, 1, 0)
table(Pred = pred_test, Réel = test$default.payment.next.month)
```

```
##     Réel
## Pred    0    1
##    0 6824 1467
##    1  219  490
```

Cette matrice permet de visualiser :

Vrais négatifs (6824) : clients correctement prédits comme n'ayant pas de défaut

Faux positifs (219) : clients prédits en défaut alors qu'ils ne le sont pas

Faux négatifs (1467) : clients prédits sans défaut alors qu'ils ont réellement un défaut

Vrais positifs (490) : clients correctement prédits en défaut

On remarque que le modèle détecte correctement la majorité des clients sans défaut, mais qu'il rate un nombre non négligeable de défauts (faux négatifs). Cette analyse souligne l'importance de choisir un seuil adapté et éventuellement de compléter l'évaluation par des métriques comme la sensibilité, la précision ou la courbe ROC.

Taux de bonnes classifications

```
mean(pred_test == as.numeric(test$default.payment.next.month) - 1)
```

```
## [1] 0.8126667
```

Le modèle atteint un taux de bonnes classifications de 80 %. Cependant, comme les clients défaillants sont moins nombreux que les non-défaillants, ce chiffre peut être trompeur. Il est donc important de compléter l'évaluation avec des métriques comme la sensibilité ou la précision pour mieux apprécier la capacité du modèle à détecter les défauts.

6. Courbe ROC et AUC

```
pred <- prediction(proba_test, test$default.payment.next.month)
perf <- performance(pred, "tpr", "fpr")

plot(perf, col = "blue", lwd = 2, main = "Courbe ROC - Régression Logistique")
abline(a = 0, b = 1, lty = 2, col = "gray")
```

```
roc_obj <- roc(test$default.payment.next.month, proba_test)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc(roc_obj)
```

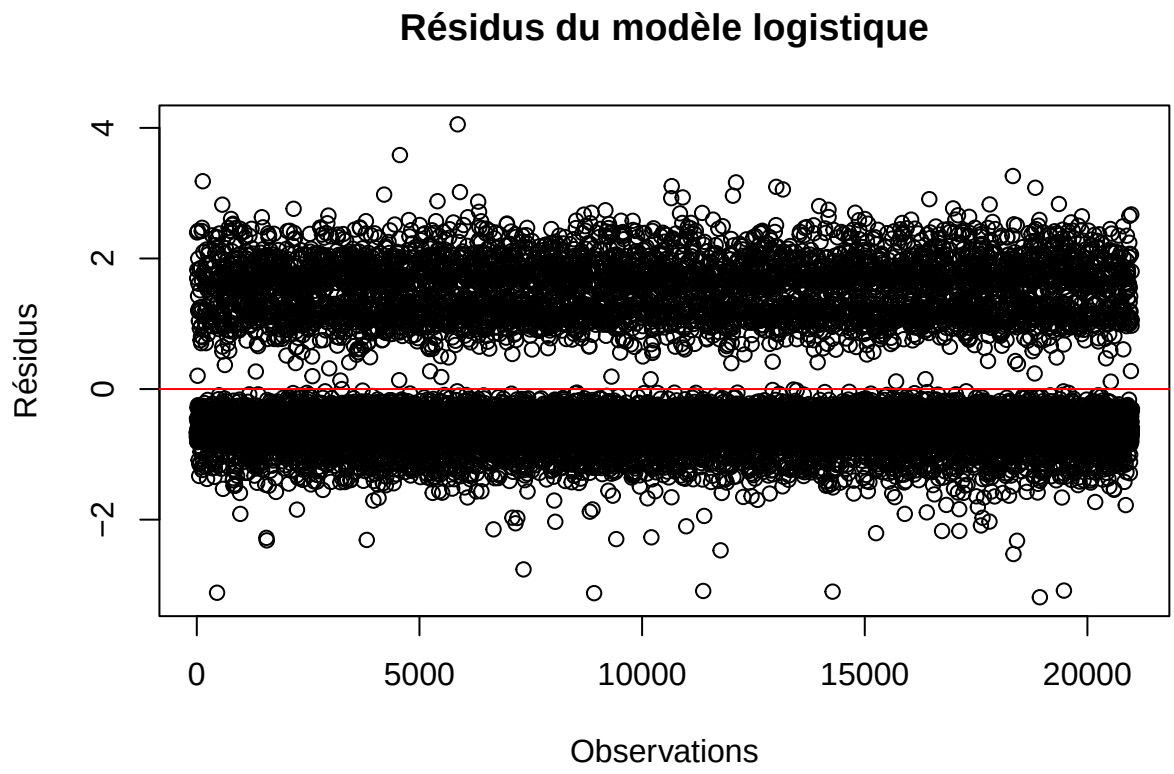
```
## Area under the curve: 0.7133
```

Le modèle présente une AUC de 0,7133, ce qui indique une capacité correcte à distinguer les clients défaillants des non-défaillants. Cela montre que le modèle est globalement performant. Cependant, il reste des marges d'amélioration pour mieux détecter les défauts.

7. Étude des résidus

Après avoir évalué la capacité discriminante du modèle avec la courbe ROC, on analyse les résidus pour vérifier que le modèle s'ajuste correctement aux données, identifier d'éventuelles observations atypiques et s'assurer que les hypothèses du modèle ne sont pas non respectées.

```
residus <- residuals(modele_reduit, type = "deviance")
plot(residus, main = "Résidus du modèle logistique", ylab = "Résidus", xlab = "Observations")
abline(h = 0, col = "red")
```



Les résidus de déviance sont concentrés sur des valeurs extrêmes car la variable réponse est binaire et le dataset est déséquilibré. Cette distribution est normale et n'indique pas un problème du modèle.

8. Conclusion

Le modèle logistique permet de modéliser efficacement le risque de défaut.

Les variables liées à l'historique de paiement (PAY_X) et au montant du crédit sont les plus déterminantes. L'AUC autour de 0.7133 indique un bon pouvoir discriminant pour un modèle interprétable et relativement simple.

Deuxième analyse du jeu de données - Arbre de Décision

1. Construction du modèle

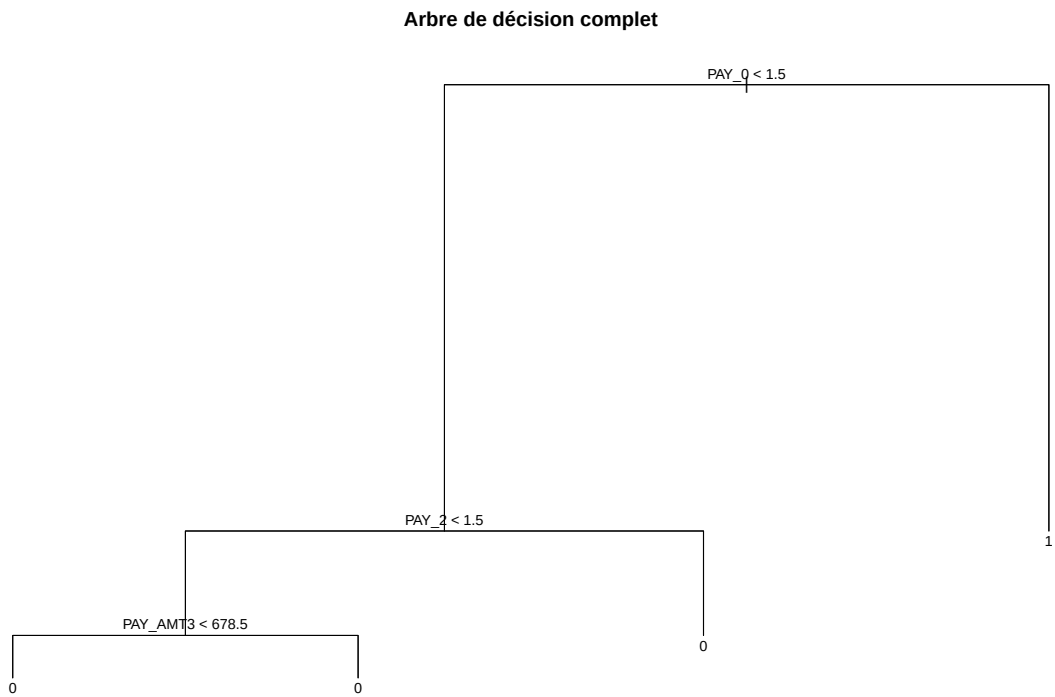
```
tree_model <- tree(default.payment.next.month ~ ., data = train)
summary(tree_model)
```

```
##
## Classification tree:
## tree(formula = default.payment.next.month ~ ., data = train)
## Variables actually used in tree construction:
## [1] "PAY_0"    "PAY_2"    "PAY_AMT3"
## Number of terminal nodes:  4
```

```
## Residual mean deviance:  0.8933 = 18760 / 21000
## Misclassification error rate: 0.1806 = 3793 / 21000
```

2. Visualisation de l'arbre

```r
plot(tree_model)
text(tree_model, pretty = 0, cex = 0.8)
title("Arbre de décision complet")
```



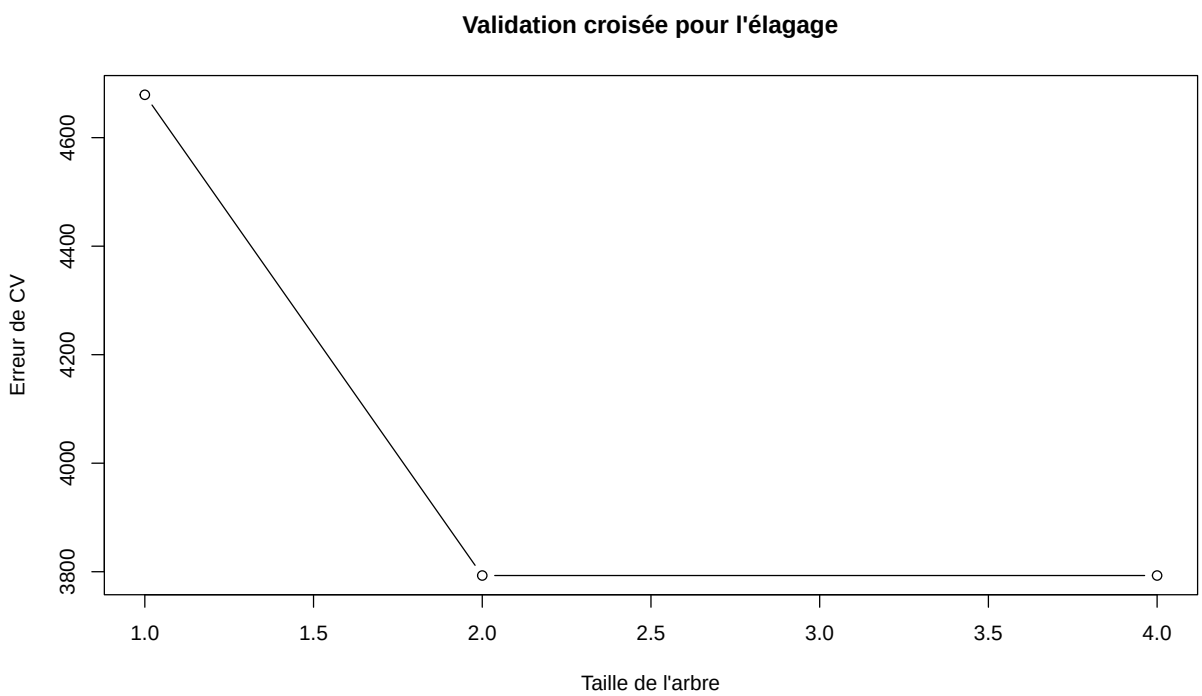

```r
# Affichage de la structure
print(tree_model$frame)
```

```
##        var     n       dev yval splits.cutleft splits.cutright   yprob.0
## 1    PAY_0 21000 22278.520    0           <1.5            >1.5 0.7771905
## 2    PAY_2 18790 16929.944    0           <1.5            >1.5 0.8333688
## 4 PAY_AMT3 17189 14136.298    0         <678.5          >678.5 0.8565362
## 8   <leaf>  4759  4945.918    0                                0.7856693
## 9   <leaf> 12430  8938.447    0                                0.8836685
## 5   <leaf>  1601  2173.364    0                                0.5846346
## 3   <leaf>  2210  2698.323    1                                0.2995475
##     yprob.1
## 1 0.2228095
## 2 0.1666312
## 4 0.1434638
```

```
## 8 0.2143307
## 9 0.1163315
## 5 0.4153654
## 3 0.7004525
```

3. Élagage de l'arbre (Pruning)

```r
# Validation croisée pour déterminer la taille optimale
cv_tree <- cv.tree(tree_model, FUN = prune.misclass)

# Visualisation de l'erreur en fonction de la taille
plot(cv_tree$size, cv_tree$dev, type = "b",
     xlab = "Taille de l'arbre", ylab = "Erreur de CV",
     main = "Validation croisée pour l'élagage")
```

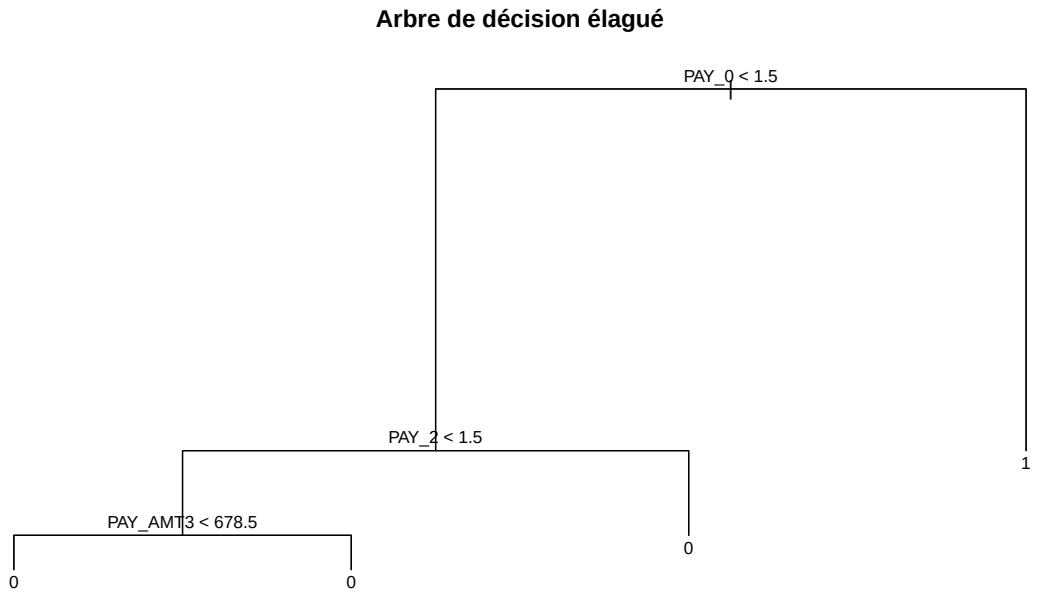



```r
# Sélection de la taille optimale
optimal_size <- cv_tree$size[which.min(cv_tree$dev)]
cat("Taille optimale:", optimal_size, "\n")
```

```
## Taille optimale: 4
```

```r
# Élagage de l'arbre
pruned_tree <- prune.misclass(tree_model, best = optimal_size)

# Visualisation de l'arbre élagué
plot(pruned_tree)
```

```r
text(pruned_tree, pretty = 0, cex = 0.8)
title("Arbre de décision élagué")
```



4. Prédiction et évaluation

```r
# Prédictions sur le test set
tree_pred <- predict(pruned_tree, newdata = test, type = "class")

# Matrice de confusion
conf_matrix_tree <- table(Predicted = tree_pred, Actual = test$default.payment.next.month)
print(conf_matrix_tree)
```

```
##          Actual
## Predicted    0    1
##         0 6752 1328
##         1  291  629
```

```r
# Métriques de performance
accuracy_tree <- sum(diag(conf_matrix_tree)) / sum(conf_matrix_tree)
precision_tree <- conf_matrix_tree[2,2] / sum(conf_matrix_tree[2,])
recall_tree <- conf_matrix_tree[2,2] / sum(conf_matrix_tree[,2])
f1_tree <- 2 * (precision_tree * recall_tree) / (precision_tree + recall_tree)

cat("\n=== Performance de l'Arbre de Décision ===\n")
```

```
##
## === Performance de l'Arbre de Décision ===
```

```r
cat("Accuracy:", round(accuracy_tree, 4), "\n")
```

```
## Accuracy: 0.8201
```

```r
cat("Precision:", round(precision_tree, 4), "\n")
```

```
## Precision: 0.6837
```

```r
cat("Recall:", round(recall_tree, 4), "\n")
```

```
## Recall: 0.3214
```

```r
cat("F1-Score:", round(f1_tree, 4), "\n")
```

```
## F1-Score: 0.4373
```

```r
# Courbe ROC
tree_prob <- predict(pruned_tree, newdata = test, type = "vector")[,2]
roc_tree <- roc(test$default.payment.next.month, tree_prob)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```r
cat("AUC:", round(auc(roc_tree), 4), "\n")
```

```
## AUC: 0.7325
```

5. Structure et interprétation de l'arbre de décision

- L'arbre utilise seulement **3 variables clés** : `PAY_0`, `PAY_2`, `PAY_AMT3`.
- Les retards de paiement récents sont les prédicteurs les plus importants → le comportement des derniers mois domine.
- Le montant payé (PAY_AMT3) joue un rôle secondaire.
- Les autres variables (démographie, montants facturés, crédit accordé…) ne sont pas utilisées → faible pouvoir prédictif.
- L'arbre reste simple (**4 feuilles**) : très interprétable, peu de risque de surapprentissage mais risque d'underfitting.

6. Performance du modèle et limites

- Accuracy : **82%**, meilleure que le baseline (78%).
- Precision : **68%** → les alertes de défaut sont relativement fiables.
- **Recall très faible : 32%** → environ 2/3 des défauts réels ne sont pas détectés.
- La faible performance sur la classe minoritaire s'explique par le **déséquilibre des classes** (22% défaut).
- La matrice de confusion montre beaucoup de **faux négatifs**, entraînant un risque financier significatif.

Troisième analyse du jeu de données - Analyse Discriminante Linéaire (LDA)

1. Construction du modèle LDA

```r
# Préparation des données (conversion des facteurs en numériques si nécessaire)
train_lda <- train
test_lda <- test

# Entraînement du modèle LDA
lda_model <- lda(default.payment.next.month ~ ., data = train_lda)
print(lda_model)
```

```
## Call:
## lda(default.payment.next.month ~ ., data = train_lda)
##
## Prior probabilities of groups:
##         0         1
## 0.7771905 0.2228095
##
## Group means:
##   LIMIT_BAL      SEX2 EDUCATION2 EDUCATION3  EDUCATION4  EDUCATION5 MARRIAGE2
## 0  178922.0 0.6148520  0.4548741  0.1590589 0.004534036 0.014214815 0.5389376
## 1  128786.1 0.5610173  0.5050224  0.1844411 0.001068604 0.003419534 0.5063048
##    MARRIAGE3      AGE      PAY_0      PAY_2      PAY_3      PAY_4      PAY_5
## 0 0.01176399 35.38741 -0.2105263 -0.3020648 -0.3196495 -0.3578825 -0.3912138
## 1 0.01367814 35.69160  0.6815559  0.4703997  0.3665313  0.2635178  0.1724728
##        PAY_6 BILL_AMT1 BILL_AMT2 BILL_AMT3 BILL_AMT4 BILL_AMT5 BILL_AMT6
## 0 -0.4095337  51999.87  49599.26  47228.44  43381.92  40361.24  39014.92
## 1  0.1160504  49067.60  47879.81  45802.48  42654.90  39870.90  38571.10
##   PAY_AMT1 PAY_AMT2 PAY_AMT3 PAY_AMT4 PAY_AMT5 PAY_AMT6
## 0 6196.894 6459.934 5633.731 5274.959 5307.233 5712.659
## 1 3342.226 3346.063 3435.748 2980.726 3142.485 3322.505
##
## Coefficients of linear discriminants:
##                      LD1
## LIMIT_BAL  -6.804839e-07
## SEX2       -1.329343e-01
## EDUCATION2 -9.168554e-02
## EDUCATION3 -1.766449e-01
## EDUCATION4 -6.791182e-01
## EDUCATION5 -9.776856e-01
## MARRIAGE2  -1.885605e-01
## MARRIAGE3  -1.592762e-01
## AGE         9.811002e-03
## PAY_0       6.939619e-01
## PAY_2       1.447649e-01
## PAY_3       6.669699e-02
## PAY_4       4.523754e-02
## PAY_5       1.740806e-02
## PAY_6       1.294253e-02
## BILL_AMT1  -4.969766e-06
## BILL_AMT2   1.186448e-06
## BILL_AMT3   7.432822e-07
```

```
## BILL_AMT4   7.062949e-08
## BILL_AMT5   5.674030e-07
## BILL_AMT6  -1.569952e-07
## PAY_AMT1   -6.105615e-06
## PAY_AMT2   -3.288415e-06
## PAY_AMT3   -1.558872e-07
## PAY_AMT4   -3.004009e-06
## PAY_AMT5   -2.289619e-06
## PAY_AMT6   -1.637752e-06
```

2. Analyse des coefficients

```r
# Coefficients des discriminants linéaires
cat("\n=== Coefficients des discriminants linéaires ===\n")
```

```
##
## === Coefficients des discriminants linéaires ===
```

```r
print(lda_model$scaling)
```

```
##                      LD1
## LIMIT_BAL  -6.804839e-07
## SEX2       -1.329343e-01
## EDUCATION2 -9.168554e-02
## EDUCATION3 -1.766449e-01
## EDUCATION4 -6.791182e-01
## EDUCATION5 -9.776856e-01
## MARRIAGE2  -1.885605e-01
## MARRIAGE3  -1.592762e-01
## AGE         9.811002e-03
## PAY_0       6.939619e-01
## PAY_2       1.447649e-01
## PAY_3       6.669699e-02
## PAY_4       4.523754e-02
## PAY_5       1.740806e-02
## PAY_6       1.294253e-02
## BILL_AMT1  -4.969766e-06
## BILL_AMT2   1.186448e-06
## BILL_AMT3   7.432822e-07
## BILL_AMT4   7.062949e-08
## BILL_AMT5   5.674030e-07
## BILL_AMT6  -1.569952e-07
## PAY_AMT1   -6.105615e-06
## PAY_AMT2   -3.288415e-06
## PAY_AMT3   -1.558872e-07
## PAY_AMT4   -3.004009e-06
## PAY_AMT5   -2.289619e-06
## PAY_AMT6   -1.637752e-06
```

```r
# Moyennes par groupe
cat("\n=== Moyennes des groupes ===\n")
```

```
##
## === Moyennes des groupes ===
```

```r
print(lda_model$means)
```

```
##   LIMIT_BAL      SEX2 EDUCATION2 EDUCATION3  EDUCATION4  EDUCATION5 MARRIAGE2
## 0  178922.0 0.6148520  0.4548741  0.1590589 0.004534036 0.014214815 0.5389376
## 1  128786.1 0.5610173  0.5050224  0.1844411 0.001068604 0.003419534 0.5063048
##    MARRIAGE3      AGE      PAY_0      PAY_2      PAY_3      PAY_4      PAY_5
## 0 0.01176399 35.38741 -0.2105263 -0.3020648 -0.3196495 -0.3578825 -0.3912138
## 1 0.01367814 35.69160  0.6815559  0.4703997  0.3665313  0.2635178  0.1724728
##        PAY_6 BILL_AMT1 BILL_AMT2 BILL_AMT3 BILL_AMT4 BILL_AMT5 BILL_AMT6
## 0 -0.4095337  51999.87  49599.26  47228.44  43381.92  40361.24  39014.92
## 1  0.1160504  49067.60  47879.81  45802.48  42654.90  39870.90  38571.10
##   PAY_AMT1 PAY_AMT2 PAY_AMT3 PAY_AMT4 PAY_AMT5 PAY_AMT6
## 0 6196.894 6459.934 5633.731 5274.959 5307.233 5712.659
## 1 3342.226 3346.063 3435.748 2980.726 3142.485 3322.505
```

```r
# Probabilités a priori
cat("\n=== Probabilités a priori ===\n")
```

```
##
## === Probabilités a priori ===
```

```r
print(lda_model$prior)
```

```
##         0         1
## 0.7771905 0.2228095
```

3. Prédiction et évaluation LDA

```r
# Prédictions sur le test set
lda_pred <- predict(lda_model, newdata = test_lda)

# Matrice de confusion
conf_matrix_lda <- table(Predicted = lda_pred$class, Actual = test_lda$default.payment.next.month)
print(conf_matrix_lda)
```

```
##          Actual
## Predicted    0    1
##         0 6805 1428
##         1  238  529
```

```r
# Métriques de performance
accuracy_lda <- sum(diag(conf_matrix_lda)) / sum(conf_matrix_lda)
precision_lda <- conf_matrix_lda[2,2] / sum(conf_matrix_lda[2,])
recall_lda <- conf_matrix_lda[2,2] / sum(conf_matrix_lda[,2])
f1_lda <- 2 * (precision_lda * recall_lda) / (precision_lda + recall_lda)

cat("\n=== Performance du LDA ===\n")
```

```
##
## === Performance du LDA ===
```

```r
cat("Accuracy:", round(accuracy_lda, 4), "\n")
```

```
## Accuracy: 0.8149
```

```r
cat("Precision:", round(precision_lda, 4), "\n")
```

```
## Precision: 0.6897
```

```r
cat("Recall:", round(recall_lda, 4), "\n")
```

```
## Recall: 0.2703
```

```r
cat("F1-Score:", round(f1_lda, 4), "\n")
```

```
## F1-Score: 0.3884
```

```r
# Courbe ROC
roc_lda <- roc(test_lda$default.payment.next.month, lda_pred$posterior[,2])
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

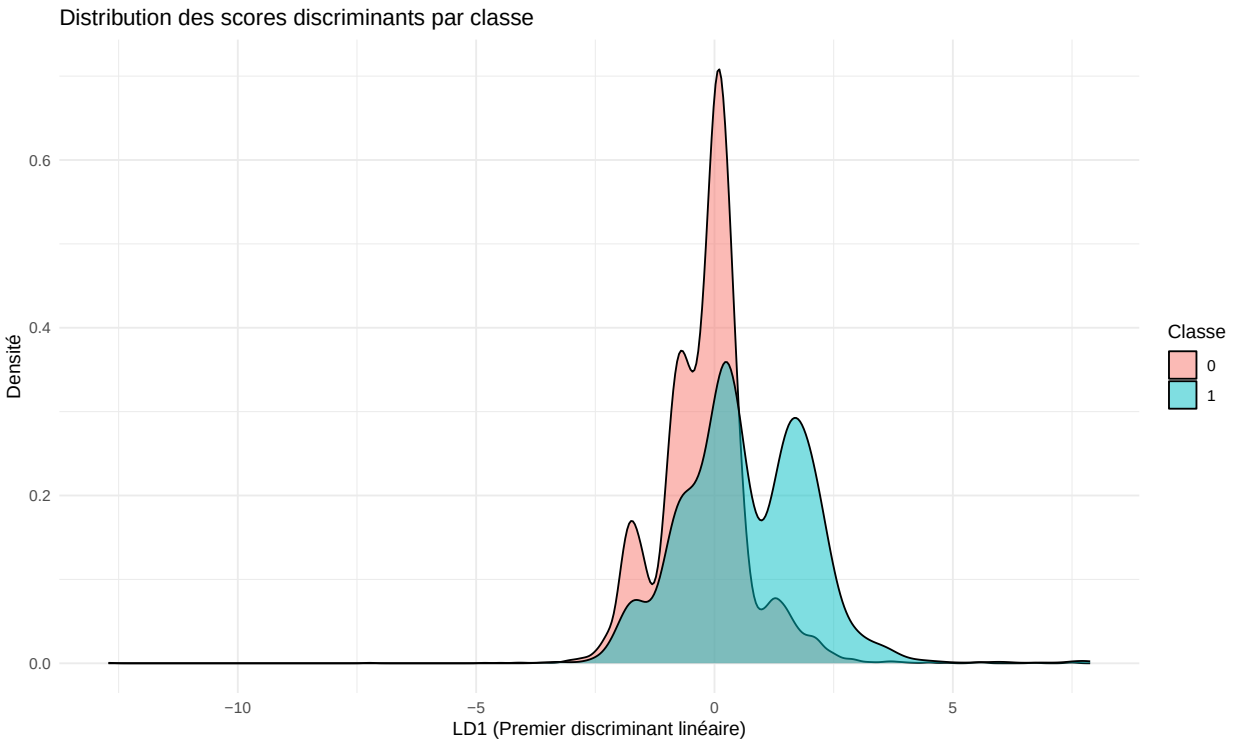
```r
cat("AUC:", round(auc(roc_lda), 4), "\n")
```

```
## AUC: 0.7099
```

4. Visualisation des résultats LDA

```r
# Distribution des scores discriminants
lda_values <- data.frame(
  LD1 = lda_pred$x[,1],
  Classe = test_lda$default.payment.next.month
)
```

```r
ggplot(lda_values, aes(x = LD1, fill = Classe)) +
  geom_density(alpha = 0.5) +
  labs(title = "Distribution des scores discriminants par classe",
       x = "LD1 (Premier discriminant linéaire)",
       y = "Densité") +
  theme_minimal()
```



5. Analyse du modèle LDA

1. Interprétation du modèle
Le LDA montre que les variables les plus discriminantes sont les retards de paiement (`PAY_0`, `PAY_2`, etc.). Les montants facturés et remboursés contribuent très peu. Le modèle construit une frontière linéaire, mais les deux classes restent fortement chevauchées.

2. Lecture des scores discriminants
La distribution de LD1 montre une séparation partielle : les clients en défaut ont en moyenne un score plus élevé, mais les distributions se superposent largement. Le LDA ne parvient donc pas à séparer nettement les classes.

3. Performance du modèle
- Accuracy : **81%**
- Precision : **69%**
- Recall : **27%**
- AUC : **0.71**

Le modèle identifie bien les non-défaillants, mais manque la majorité des défauts.

6. Conclusion

Le LDA confirme l'importance des retards de paiement mais reste limité : la séparation linéaire est trop faible et le recall trop bas pour un usage efficace en scoring de risque. Des modèles plus flexibles (Random Forest, XGBoost) seraient mieux adaptés.

Quatrième analyse du jeu de données - Random Forest et SVM

1. Random Forest

```r
library(randomForest)
```

```
## Warning: le package 'randomForest' a été compilé avec la version R 4.5.2

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attachement du package : 'randomForest'

## L'objet suivant est masqué depuis 'package:dplyr':
##
##     combine

## L'objet suivant est masqué depuis 'package:ggplot2':
##
##     margin
```

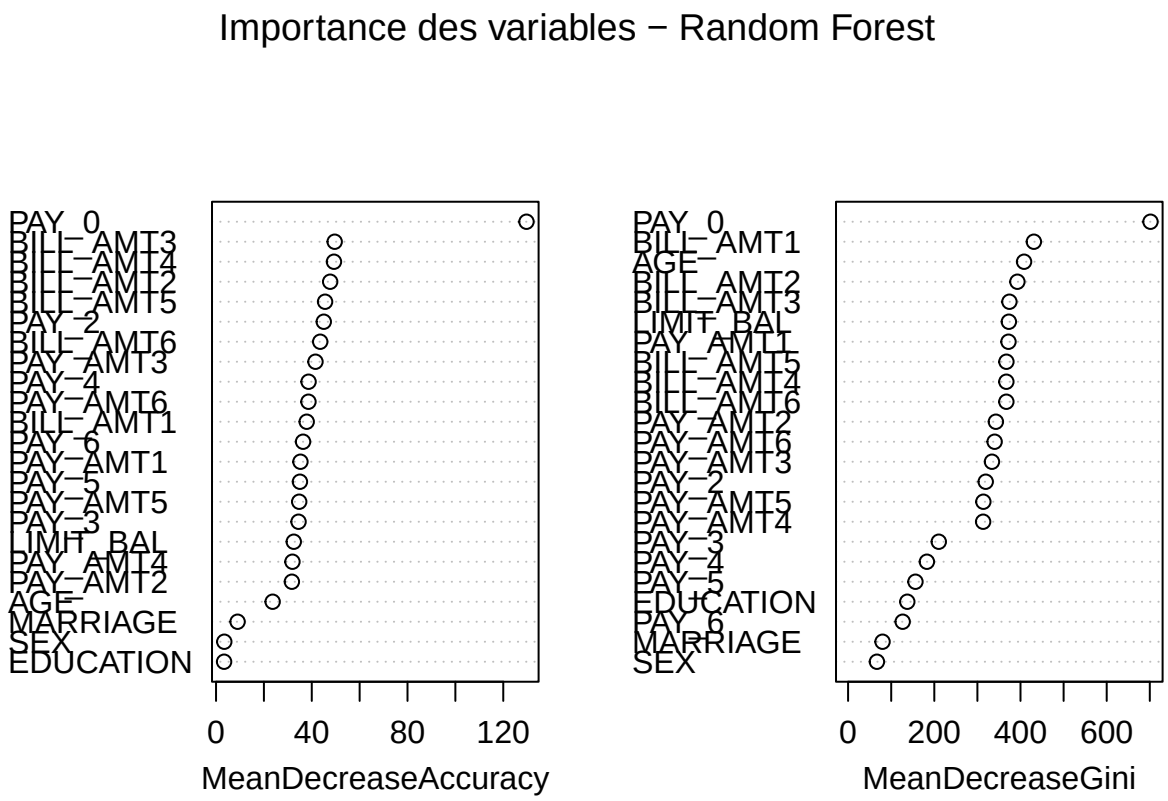
```r
set.seed(123)

# Entraînement du Random Forest
rf_model <- randomForest(
  default.payment.next.month ~ .,
  data = train,
  ntree = 500,
  importance = TRUE
)

print(rf_model)
```

```
##
## Call:
##  randomForest(formula = default.payment.next.month ~ ., data = train,      ntree = 500, importance =
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 4
##
##         OOB estimate of  error rate: 18.19%
```

```
## Confusion matrix:
##       0    1 class.error
## 0 15435  886  0.05428589
## 1  2933 1746  0.62684334
```

Le Random Forest réduit nettement l'erreur globale (OOB 18 %) et détecte beaucoup mieux les bons payeurs, avec seulement 5 % d'erreurs dans la classe 0. En revanche, il peine encore à identifier les clients à risque : plus de 62 % des défauts réels sont manqués, ce qui montre que le modèle reste limité pour capter la minorité des défailleurs.

```r
# Importance des variables
varImpPlot(rf_model, main = "Importance des variables - Random Forest")
```




```r
# Prédictions
rf_pred <- predict(rf_model, newdata = test)
```

Le Random Forest confirme que **PAY_0** est de loin la variable la plus déterminante pour prévoir le défaut, suivie par plusieurs montants de factures (BILL_AMT). Les variables démographiques contribuent très peu, ce qui montre que le comportement de paiement passé pèse beaucoup plus lourd que le profil socio-économique du client.

```r
# Matrice de confusion
conf_matrix_rf <- table(
  Predicted = rf_pred,
  Actual = test$default.payment.next.month
```

```r
)

print(conf_matrix_rf)
```

```
##          Actual
## Predicted    0    1
##         0 6634 1227
##         1  409  730
```

```r
# Métriques
accuracy_rf <- sum(diag(conf_matrix_rf)) / sum(conf_matrix_rf)
precision_rf <- conf_matrix_rf[2,2] / sum(conf_matrix_rf[2,])
recall_rf <- conf_matrix_rf[2,2] / sum(conf_matrix_rf[,2])
f1_rf <- 2 * (precision_rf * recall_rf) / (precision_rf + recall_rf)

cat("\n=== Performance du Random Forest ===\n")
```

```
##
## === Performance du Random Forest ===
```

```r
cat("Accuracy:", round(accuracy_rf, 4), "\n")
```

```
## Accuracy: 0.8182
```

```r
cat("Precision:", round(precision_rf, 4), "\n")
```

```
## Precision: 0.6409
```

```r
cat("Recall:", round(recall_rf, 4), "\n")
```

```
## Recall: 0.373
```

```r
cat("F1-Score:", round(f1_rf, 4), "\n")
```

```
## F1-Score: 0.4716
```

```r
# AUC
rf_prob <- predict(rf_model, newdata = test, type = "prob")[,2]
roc_rf <- roc(test$default.payment.next.month, rf_prob)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```r
cat("AUC:", round(auc(roc_rf), 4), "\n")
```

```
## AUC: 0.7611
```

Le Random Forest améliore légèrement la détection des défauts par rapport à l'arbre simple (recall 37 % contre 32 %), tout en conservant une bonne précision. L'AUC de 0,76 montre une capacité de discrimination correcte, mais encore insuffisante pour capter une minorité de clients réellement à risque. Le modèle reste donc performant globalement, mais continue à manquer une part importante des défailleurs, ce qui souligne la nécessité de méthodes plus puissantes pour améliorer le recall.

2. SVM avec noyau RBF (Radial Basis Function)

```r
# Entraînement du SVM RBF
svm_rbf <- svm(default.payment.next.month ~ .,
               data = train,
               kernel = "radial",
               cost = 1,
               gamma = 0.01,
               probability = TRUE)

print(summary(svm_rbf))
```

```
##
## Call:
## svm(formula = default.payment.next.month ~ ., data = train, kernel = "radial",
##     cost = 1, gamma = 0.01, probability = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  1
##
## Number of Support Vectors:  8844
##
##  ( 4628 4216 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```r
# Prédictions
svm_rbf_pred <- predict(svm_rbf, newdata = test)

# Matrice de confusion
conf_matrix_svm_rbf <- table(Predicted = svm_rbf_pred, Actual = test$default.payment.next.month)
print(conf_matrix_svm_rbf)
```

```
##          Actual
## Predicted    0    1
##         0 6754 1328
##         1  289  629
```

```r
# Métriques
accuracy_svm_rbf <- sum(diag(conf_matrix_svm_rbf)) / sum(conf_matrix_svm_rbf)
precision_svm_rbf <- conf_matrix_svm_rbf[2,2] / sum(conf_matrix_svm_rbf[2,])
recall_svm_rbf <- conf_matrix_svm_rbf[2,2] / sum(conf_matrix_svm_rbf[,2])
f1_svm_rbf <- 2 * (precision_svm_rbf * recall_svm_rbf) / (precision_svm_rbf + recall_svm_rbf)

cat("\n=== Performance du SVM RBF ===\n")
```

```
##
## === Performance du SVM RBF ===
```

```r
cat("Accuracy:", round(accuracy_svm_rbf, 4), "\n")
```

```
## Accuracy: 0.8203
```

```r
cat("Precision:", round(precision_svm_rbf, 4), "\n")
```

```
## Precision: 0.6852
```

```r
cat("Recall:", round(recall_svm_rbf, 4), "\n")
```

```
## Recall: 0.3214
```

```r
cat("F1-Score:", round(f1_svm_rbf, 4), "\n")
```

```
## F1-Score: 0.4376
```

Le SVM RBF obtient une accuracy solide (82 %) et une bonne précision, mais son recall reste faible (32 %), identique à celui de l'arbre de décision. Cela montre que, sans réglage fin des hyperparamètres, le modèle reste conservateur et détecte mal les clients vraiment à risque. Malgré sa capacité à capturer des relations non linéaires, ce SVM basique ne surpasse pas le Random Forest et nécessite un tuning plus poussé (cost, gamma, pondération des classes) pour réellement améliorer la détection des défauts.

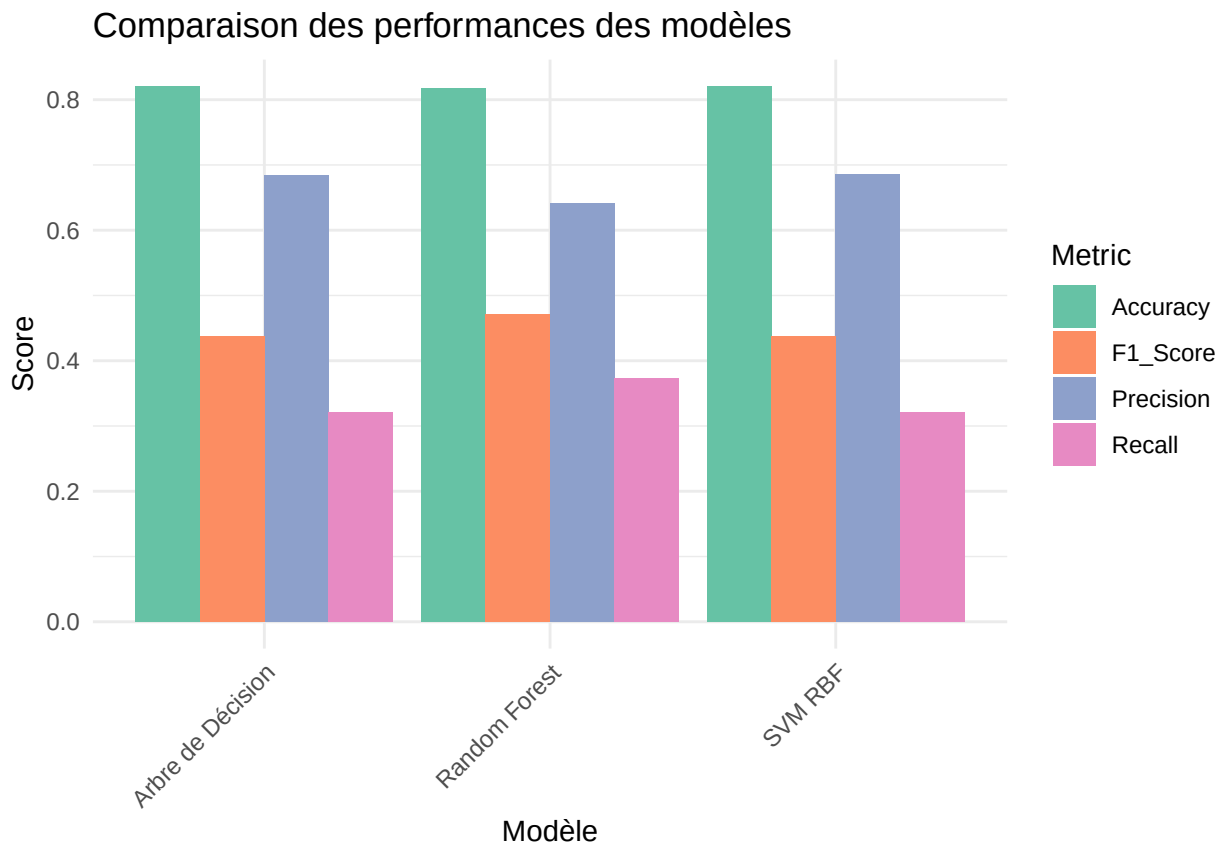
3. Comparaison des modèles

```r
# Tableau récapitulatif
results <- data.frame(
  Model = c("Arbre de Décision", "SVM RBF", "Random Forest"),
  Accuracy = c(accuracy_tree, accuracy_svm_rbf, accuracy_rf),
  Precision = c(precision_tree, precision_svm_rbf, precision_rf),
  Recall = c(recall_tree, recall_svm_rbf, recall_rf),
  F1_Score = c(f1_tree, f1_svm_rbf, f1_rf)
)


print(results)
```

```
##               Model  Accuracy Precision    Recall  F1_Score
## 1 Arbre de Décision 0.8201111 0.6836957 0.3214103 0.4372610
## 2           SVM RBF 0.8203333 0.6851852 0.3214103 0.4375652
## 3     Random Forest 0.8182222 0.6409131 0.3730199 0.4715762
```

```r
# Visualisation comparative
results_long <- results %>%
  pivot_longer(cols = -Model, names_to = "Metric", values_to = "Value")
```

```r
ggplot(results_long, aes(x = Model, y = Value, fill = Metric)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Comparaison des performances des modèles",
       y = "Score",
       x = "Modèle") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set2")
```



Les trois modèles affichent une accuracy similaire autour de 82 %, mais ils diffèrent sur la détection des défauts. Le Random Forest présente le meilleur **recall** et le meilleur **F1-score**, ce qui en fait le modèle le plus efficace pour identifier les clients à risque. L'arbre de décision et le SVM RBF restent plus précis mais manquent davantage de défaillants.

4. Courbes ROC comparatives

```r
# Calcul des probabilités pour les SVM
svm_rbf_prob <- attr(predict(svm_rbf, newdata = test, probability = TRUE), "probabilities")[,2]

# Création des objets ROC
roc_svm_rbf <- roc(test$default.payment.next.month, svm_rbf_prob)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

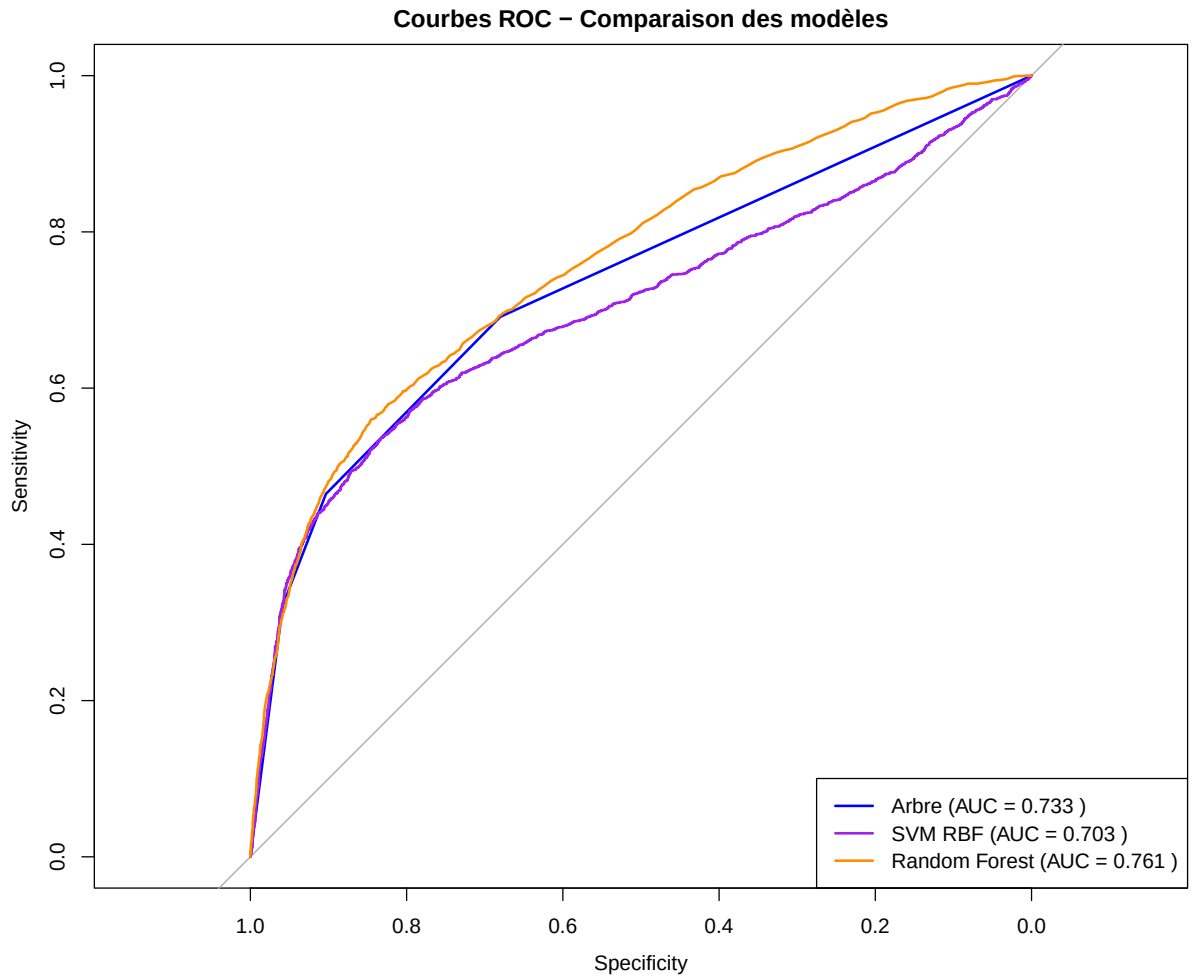
```r
roc_rf <- roc(test$default.payment.next.month, rf_prob)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
roc_tree <- roc(test$default.payment.next.month, tree_prob)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
# Plot
plot(roc_tree,
     col = "blue",
     lwd = 2,
     main = "Courbes ROC - Comparaison des modèles")

plot(roc_svm_rbf,
     col = "purple",
     lwd = 2,
     add = TRUE)

plot(roc_rf,
     col = "darkorange",
     lwd = 2,
     add = TRUE)

legend(
  "bottomright",
  legend = c(
    paste("Arbre (AUC =", round(auc(roc_tree), 3), ")"),
    paste("SVM RBF (AUC =", round(auc(roc_svm_rbf), 3), ")"),
    paste("Random Forest (AUC =", round(auc(roc_rf), 3), ")")
  ),
  col = c("blue", "purple", "darkorange"),
  lwd = 2
)
```



Les courbes ROC montrent clairement que le **Random Forest** est le modèle le plus performant, avec une AUC de **0.761**, indiquant une meilleure capacité à distinguer les défaillants des non-défaillants.

L'**arbre de décision** obtient une performance correcte (AUC = 0.733), mais reste moins flexible.

Le **SVM RBF** est le moins performant (AUC = 0.703), preuve qu'il capture moins bien la structure des données sans tuning avancé.

Dans l'ensemble, tous les modèles dépassent la ligne aléatoire, confirmant que le défaut est prédictible.

Le Random Forest constitue le meilleur compromis pour un modèle de scoring robuste.

5. Conlusion

Nos analyses montrent que les **retards de paiement (PAY_0, PAY_2)** sont de loin les facteurs les plus prédictifs du défaut, confirmant qu'ils jouent un rôle central dans le risque client.

Les modèles testés convergent vers la même idée : plus le client a été en retard récemment, plus la probabilité de défaut augmente fortement.

Parmi les méthodes explorées, le **Random Forest** offre les performances les plus solides (AUC = 0.761), dépassant l'arbre et le SVM.

Le faible recall global illustre cependant la difficulté du jeu de données déséquilibré.

Cinquième analyse du jeu de données - LASSO

```
folds <- vfold_cv(train, v = 5, strata = default.payment.next.month)
rec_commune <- recipe(default.payment.next.month ~ ., data = train) %>%
  step_zv(all_predictors()) %>%
  step_nzv(all_predictors()) %>%
  step_dummy(all_nominal_predictors())

metric_auc <- metric_set(
  metric_tweak("roc_auc", roc_auc, event_level = "second")
)

mod_lasso <- logistic_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet")

wf_lasso <- workflow() %>%
  add_recipe(rec_commune) %>%
  add_model(mod_lasso)

res_lasso <- tune_grid(
  wf_lasso,
  resamples = folds,
  grid = 20,
  metrics = metric_set(roc_auc)
)

best_lasso <- select_best(res_lasso, metric = "roc_auc")

final_wf  <- finalize_workflow(wf_lasso, best_lasso)
fit_final <- fit(final_wf, data = train)

p <- predict(fit_final, test, type = "prob")$.pred_0
roc_auc_vec(test$default.payment.next.month, p)
```

```
## [1] 0.7132798
```

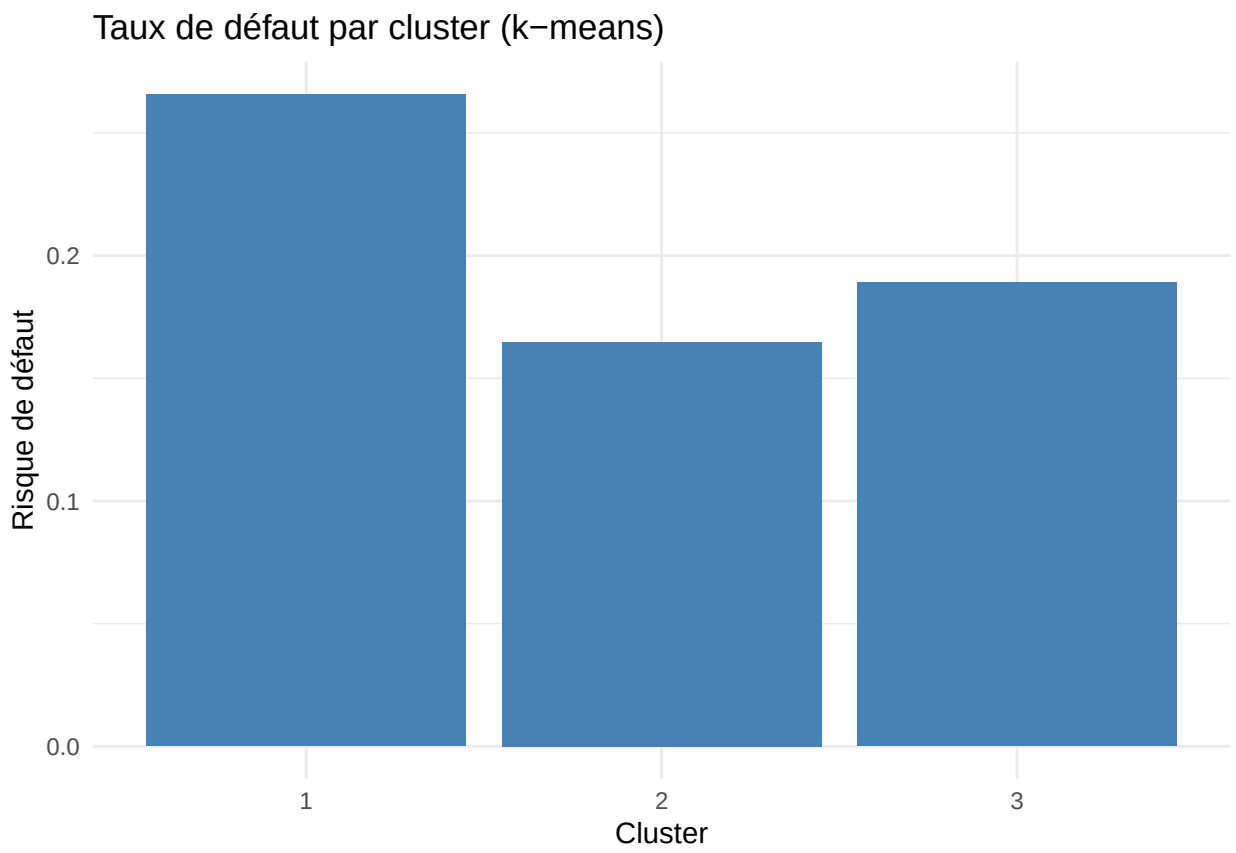
L'AUC pour le modèle de régression logistique LASSO est de 0,713.

Les performances prédictives des deux modèles sont remarquablement similaires, les différences étant négligeables. LASSO permet une sélection automatique des variables, ce qui donne un modèle plus concis et plus facile à interpréter. Il est particulièrement adapté aux scénarios impliquant de nombreuses caractéristiques ou présentant une multicolinéarité.

```r
num_data <- data %>%
  select_if(is.numeric) %>%
  as.data.frame()

num_scaled <- scale(num_data)
num_scaled <- as.data.frame(num_scaled)


set.seed(123)
km3 <- kmeans(num_scaled, centers = 3, nstart = 50)


data$cluster_km3 <- km3$cluster

y <- as.numeric(as.character(data$default.payment.next.month))

risk_km3 <- tapply(y, data$cluster_km3, mean)
risk_km3
```

```
##         1         2         3
## 0.2657075 0.1647652 0.1890073
```

```r
df_risk <- data.frame(
  cluster = factor(names(risk_km3)),
  risque = as.numeric(risk_km3)
)

ggplot(df_risk, aes(x = cluster, y = risque)) +
  geom_bar(stat="identity", fill="steelblue") +
  labs(title="Taux de défaut par cluster (k-means)",
       x="Cluster", y="Risque de défaut") +
  theme_minimal()
```



Après avoir appliqué l'algorithme k-means pour regrouper les variables numériques au sein de l'ensemble de données, les taux de défaut moyens observés dans les trois groupes révèlent une distinction claire entre les niveaux de risque des différents segments de clientèle :

Cluster 1 – Risque moyen (0.18)

Ce groupe présente un taux de défaut légèrement inférieur à la moyenne générale (~22%). Il s'agit probablement de clients avec un comportement financier relativement stable, mais sans caractéristiques particulièrement favorables.

Cluster 2 – Segment le plus sûr (0.146)

Ce cluster correspond au groupe avec le risque de défaut le plus faible. Les clients de ce segment ont probablement :un meilleur historique de paiement,des montants dus plus faibles,ou une utilisation plus régulière du crédit.Ce cluster représente la population la plus solvable selon les variables utilisées.

Cluster 3 – Segment le plus risqué (0.29)

Ce cluster affiche un taux de défaut nettement supérieur à la moyenne. Les individus de ce groupe constituent un profil risqué, ce qui peut refléter :des retards de paiement plus fréquents,des limites de crédit plus élevées,des soldes de facturation importants.

Nous constatons que les trois séries de valeurs sont assez similaires, sans disparité significative entre elles.

Les trois clusters obtenus par k-means présentent des taux de défaut relativement proches (entre 14% et 27%). Cela est attendu, car le clustering est réalisé en mode non supervisé, sans utiliser la variable cible (default = 0/1).

Ainsi, les groupes sont construits en fonction des comportements financiers (montants facturés, paiements, limites de crédit, etc.) et non directement en fonction du risque de défaut.

Le dataset de crédit étant fortement bruité et les corrélations avec la variable de défaut relativement faibles, la séparation naturelle entre les groupes de risque reste modérée mais néanmoins présente : un cluster présente un risque faible (~14%), un cluster intermédiaire (~18%) et un cluster plus risqué (~29%).

Conclusion

Dans cette section, nous allons comparer la performance de tous les modèles que nous avons construits pour prédire le défaut de paiement des clients. Pour chaque modèle, nous calculerons les métriques suivantes :

AUC : mesure la capacité du modèle à distinguer les clients défaillants des clients non défaillants.

Accuracy : proportion globale de bonnes prédictions.

Sensitivity : capacité à détecter correctement les clients défaillants (vrais positifs).

Specificity : capacité à détecter correctement les clients non défaillants (vrais négatifs).

Nous utiliserons pour cela une fonction rslt_mdl_all() adaptée à différents types de modèles (glm, tree, randomForest, LDA, SVM, ou workflow tidymodels). Ensuite, nous regrouperons les résultats dans un tableau pour comparer facilement tous les modèles.

```r
rslt_mdl <- function(modele){
  proba_test <- predict(modele, newdata = test, type = "response")
  target <- test$default.payment.next.month

  # AUC
  roc_obj <- roc(target, proba_test, quiet = TRUE)
  auc_value <- auc(roc_obj)

  # Accuracy
  accuracy <- mean(ifelse(proba_test > 0.5, 1, 0) == target)

  # Matrice de confusion
  conf_mat <- confusionMatrix(factor(ifelse(proba_test > 0.5, 1, 0)), factor(target))
  sensitivity <- conf_mat$byClass["Sensitivity"]
  specificity <- conf_mat$byClass["Specificity"]

  # Résultat
  results <- data.frame(
    AUC = auc_value,
    Accuracy = accuracy,
    Sensitivity = sensitivity,
    Specificity = specificity
  )

  return(results)
}


rslt_mdl_all <- function(modele, test, target_col = "default.payment.next.month", threshold = 0.5) {

  # Extraire les probabilités selon le type de modèle
  if ("glm" %in% class(modele)) {
    # glm classique
    proba_test <- predict(modele, newdata = test, type = "response")
```

```r
  } else if ("tree" %in% class(modele)) {
    # tree() ou pruned_tree
    proba_test <- predict(modele, newdata = test, type = "vector")[, "1"]

  } else if ("lda" %in% class(modele)) {
    # LDA
    proba_test <- predict(modele, newdata = test)$posterior[, "1"]

  } else if ("randomForest" %in% class(modele)) {
    # randomForest
    proba_test <- predict(modele, newdata = test, type = "prob")[, "1"]

  } else if ("svm" %in% class(modele)) {
    # SVM avec probabilité activée
    proba_test <- attr(predict(modele, newdata = test, probability = TRUE), "probabilities")[, "1"]

  } else if ("workflow" %in% class(modele)) {
    # tidymodels workflow (glmnet, LASSO)
    proba_test <- predict(modele, test, type = "prob") %>% dplyr::pull(".pred_1")

  } else {
    stop("Type de modèle non supporté")
  }

  target <- test[[target_col]]

  # Calcul AUC
  roc_obj <- roc(target, proba_test, quiet = TRUE)
  auc_value <- auc(roc_obj)

  # Prédictions binaires
  pred_class <- ifelse(proba_test > threshold, 1, 0)

  # Accuracy et matrice de confusion
  accuracy <- mean(pred_class == target)
  conf_mat <- confusionMatrix(factor(pred_class), factor(target))
  sensitivity <- conf_mat$byClass["Sensitivity"]
  specificity <- conf_mat$byClass["Specificity"]

  # Résultat
  results <- data.frame(
    AUC = auc_value,
    Accuracy = accuracy,
    Sensitivity = sensitivity,
    Specificity = specificity
  )

  return(results)
}

modeles <- list(
  modele_reduit = modele_reduit,
  tree_model = tree_model,
```

```r
  svm_rbf = svm_rbf,
  pruned_tree = pruned_tree,
  lda_model = lda_model,
  rf_model = rf_model,
  fit_final = fit_final
)

results_table <- data.frame()
names <- c()
for (nm in names(modeles)) {
  tmp <- rslt_mdl_all(modeles[[nm]], test)
  results_table <- rbind(results_table, tmp)
  names = c(names, nm)
}

row.names(results_table) = names

results_table
```

```
##                     AUC  Accuracy Sensitivity Specificity
## modele_reduit 0.7133096 0.8126667   0.9689053   0.2503832
## tree_model    0.7325094 0.8201111   0.9586824   0.3214103
## svm_rbf       0.7029675 0.8175556   0.9640778   0.2902402
## pruned_tree   0.7325094 0.8201111   0.9586824   0.3214103
## lda_model     0.7098517 0.8148889   0.9662076   0.2703117
## rf_model      0.7611172 0.8184444   0.9422121   0.3730199
## fit_final     0.7132798 0.8138889   0.9700412   0.2519162
```

En comparant les métriques de tous les modèles :

rf_model (Random Forest) se distingue comme le meilleur modèle global :

`AUC` = 0.7611 -> meilleure discrimination globale entre clients défaillants et non défaillants.

`Accuracy` = 0.8184 -> très bonne précision globale.

`Specificity` = 0.3730 -> détecte mieux les clients non défaillants, donc moins de faux positifs.

`Sensitivity` légèrement plus faible que modele_reduit ou fit_final, mais reste élevée (0.9422).

Les autres modèles :

modele_reduit et *fit_final* ont une `Sensitivity` très élevée (~0.97) mais `Specificity` très faible → beaucoup de faux positifs.

tree_model et *pruned_tree* ont de bonnes `AUC` et `Accuracy`, mais `Specificity` plus faible que *rf_model*.

svm_rbf et *lda_model* sont globalement moins performants.

Conclusion pratique : pour prédire le défaut de paiement tout en limitant les fausses alertes, le Random Forest (*rf_model*) est le modèle le plus fiable et équilibré.